

Tree -1

Q.N0:1

DS

Session

Tree 1

Question Information

Level 1

Challenge 01

Question description

Jagan Sir students were chatting and playing quite loudly on the last day of the year, celebrating the end of the academic session. Jagan sir was harshly chastised by the college's principal. But, instead of becoming enraged, he attempted to engage everyone in a different task.

So Jagan sir gave his students to solve the task such that, you have to perform pre-order tree traversal in Binary search tree.

```
graph TD; 1((1)) --> 2((2)); 1 --> 3((3)); 2 --> 4((4)); 2 --> 5((5)); 4 --> 8((8)); 4 --> 9((9)); 3 --> 6((6)); 3 --> 7((7)); 6 --> 10((10)); 6 --> 11((11)); 7 --> 12((12));
```

Direction : Anti Clock

Rule : CLR (Center-Left-Right)

PreOrder:
1 2 4 8 9 5 3 6 10 7 11 12

Constraints:

0<size<1000
0<value<10000

Input format:

First line indicates the size of the tree.
Second line indicates the elements of the tree

Output format:

single line indicates the preorder list of the given value.

Logical Test Cases

Test Case 1

INPUT (STDIN)

7

200 120 80 160 280 240 320

EXPECTED OUTPUT

200 120 80 160 280 240 320

Test Case 2

INPUT (STDIN)

9

34 67 10 90 410 810 40 20 60

EXPECTED OUTPUT

34 10 20 67 40 60 90 410 810

Mandatory Test Cases

Test Case 1

KEYWORD

void preorder(struct node* root)

Test Case 2

KEYWORD

struct node* newNode(int item)

Test Case 3

KEYWORD

struct node

Test Case 4

KEYWORD

struct node *left,*right;

Complexity Test Cases

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *left, *right;
};

struct node* newNode(int item)
{
    struct node* temp;

    temp = (struct node*)malloc(sizeof(struct node));

    temp->data = item;
    temp->left = NULL;
    temp->right = NULL;

    return temp;
}

struct node* insert(struct node* root, int item)
{
    if (root == NULL)
    {
        return newNode(item);
    }

    if (item < root->data)
    {
        root->left = insert(root->left, item);
    }
    else
    {
        root->right = insert(root->right, item);
    }

    return root;
}

void preorder(struct node* root)
{
    if (root != NULL)
    {
        printf("%d ", root->data);

        preorder(root->left);

        preorder(root->right);
    }
}
```

```
int main()
{
    int n;
    int a[100];

    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    struct node* root = NULL;

    for (int i = 0; i < n; i++)
    {
        root = insert(root, a[i]);
    }

    preorder(root);

    return 0;
}
```

Output:

Test_case 1:

```
7
200 120 80 160 280 240 320
200 120 80 160 280 240 320 %
```

Test_case 2:

```
9
34 67 10 90 410 80 40 20 60
34 10 20 67 40 60 90 80 410 %
```